🎓 Placement Exam System

Developed by Team Kairo 🏆

A production-ready, enterprise-grade online placement examination platform with advanced anti-cheating mechanisms, real-time monitoring, and comprehensive code execution environment.

Key Capabilities: Per-Round Violations • 500+ Concurrent Users • Real-Time Monitoring • Live Code Execution • Advanced Proctoring

---

📋 Table of Contents

1. Executive Summary
2. System Overview
3. Core Features
4. Technical Architecture
5. Security & Proctoring
6. Technology Stack
7. Installation & Setup
8. User Guide
9. Admin Dashboard
10. Database Architecture
11. Deployment
12. Performance & Scalability
13. Project Statistics
14. Support & Documentation

📊 Executive Summary

The **Placement Exam System** is a comprehensive web-based assessment platform designed for conducting secure, scalable online placement examinations. Built with modern web technologies, it provides a robust solution for educational institutions and organizations to evaluate candidates across multiple assessment rounds.

Key Highlights

- **Multi-Round Assessment**: 3-stage evaluation (Aptitude, Core Subjects, DSA Coding)
- **Concurrent Users**: Optimized for 500+ simultaneous exam takers
- **Security**: 17+ violation detection mechanisms with per-round tracking
- **Code Execution**: Live programming evaluation across 5 languages
- **Real-Time Monitoring**: Admin dashboard with live student activity tracking
- **Production Ready**: Full Firebase integration with scalable architecture

---

🎯 System Overview

Assessment Model

The system implements a **three-round sequential assessment** designed to comprehensively evaluate candidates:

```
┌──────────────────────────────────────────────────────────────┐
│                    PLACEMENT EXAM WORKFLOW                   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ROUND 1: APTITUDE (MCQ)                                     │
│  ├─ Logical Reasoning                                        │
│  ├─ Quantitative Aptitude                                    │
│  └─ Verbal Ability                                           │
│                                                              │
│  ROUND 2: CORE SUBJECTS (MCQ)                                │
│  ├─ Database Management Systems                              │
│  ├─ Operating Systems                                        │
│  ├─ Computer Networks                                        │
│  └─ Data Structures & OOP                                    │
│                                                              │
│  ROUND 3: DSA CODING (Live Programming)                      │
│  ├─ Algorithm Implementation                                 │
│  ├─ Problem Solving                                          │
│  └─ Code Optimization                                        │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

User Roles

Role	Access Level	Capabilities
Student	Exam Participant	Take exams, view personal results, access dashboard
Admin	Full System Access	Create/manage exams, question bank management, view all submissions, monitor violations

---

✨ Core Features

1. Authentication & Access Control

- **Firebase Authentication** with email/password
- Role-based access control (RBAC)
- Secure session management with automatic token refresh
- Protected routes with role validation
- One-time exam access per student (prevents retakes)

2. Student Examination Features

Pre-Exam Phase

- Interactive instruction display with terms acknowledgment
- Exam code validation (3-attempt limit, case-insensitive)
- Dynamic countdown timer until exam start
- Browser compatibility verification
- Mandatory screen sharing enforcement
- Automatic fullscreen activation

During Exam

- **Three sequential rounds** with 30-second transition screens
- **Individual timers** per round with visual alerts
- **Question palette** showing answered/unanswered status
- **Smart navigation**: Previous, Next, Jump to Question, Clear Answer
- **Auto-save functionality** with localStorage backup
- **Per-round violation tracking** (3 violations max per round, resets between rounds)
- **Blocking violation modals** requiring acknowledgment
- **Real-time proctoring indicators**

Post-Exam

- Comprehensive score display with round-wise breakdown
- Submission timestamp and confirmation
- Violation summary per round
- Automatic data cleanup and logout

3. Admin Management System

Dashboard Overview

- Real-time system statistics (exams, students, submissions)
- Active student monitoring (last 5 minutes)
- High-violation alerts (3+ violations)
- Quick action buttons for common tasks

Exam Management

- **Exam Creation Wizard**:
  - Custom title and unique code generation
  - Start date/time scheduling
  - Individual round durations
  - Question distribution by difficulty (Easy/Medium/Hard)
  - Auto-validation against question bank
- **Exam Lifecycle Management**: View, edit, delete
- **Search and filter** capabilities

Question Bank Management

- **MCQ Questions**: Category, difficulty, 4 options, correct answer
- **DSA Coding Questions**:
  - Problem statement builder
  - Input/output examples
  - Constraints and hints
  - Starter code templates (5 languages)
  - **Test Case Manager**: Visible and hidden test case separation
- **Bulk Operations**: Delete multiple questions
- **Live Statistics**: Question count per category and difficulty
- **One-click seeding**: Pre-built DSA question set (5 problems)

Results Management

- Comprehensive submission viewer with filters
- Student performance analytics:
  - Round-wise scores
  - Time taken per round
  - Violation count per round
  - DSA code review
- Export functionality (CSV/JSON)
- Code submission viewer with execution results

Real-Time Monitoring

- **Live Activity Feed** (Firestore onSnapshot)
- **Admin Control Actions**:
  - Force submit student exam
  - Kick student from exam
  - Action logging for audit trail
- **Violation Analytics Dashboard**
- **High-risk student flagging**

4. DSA Coding Round (Round 3)

Code Editor

- **Resizable split-panel interface**:
  - Horizontal resize: Problem description ↔ Code editor
  - Vertical resize: Code editor ↔ Output console
- **5 Programming Languages**:

Language	Compiler	Judge0 ID
Python	3.8.1	71
C	GCC 9.2.0	50
C++	GCC 9.2.0	54

Language	Compiler	Judge0 ID
Java	OpenJDK 13	62
JavaScript	Node.js 12	63

- **Syntax highlighting** and auto-indentation
- **Custom input console** for testing

Execution Modes

1. **Practice Mode (Test Button)**:
    - Runs only visible test cases
    - Shows full input/output details
    - For debugging and validation
    - No score calculation
2. **Submit Mode (Submit Solution Button)**:
    - Executes ALL test cases (visible + hidden)
    - Automatic score calculation
    - Shows pass/fail status
    - One-time submission per problem
    - Saves to Firestore

Judge0 Integration

- Real-time code compilation and execution
- Asynchronous polling (1-second intervals, 30-second timeout)
- Comprehensive error handling:
    - Compilation errors with line numbers
    - Runtime errors with stack traces
    - Time Limit Exceeded (TLE)
    - Memory Limit Exceeded (MLE)
- Execution metrics: Time, Memory, Status

Grading System

```
Score = (Passed Tests / Total Tests) × 100
```

- Partial credit for partially correct solutions
- Separate tracking of visible vs hidden test results
- Automatic submission to Firestore with full details

Test Case System

- **Visible Test Cases**: Shown to students for validation

- **Hidden Test Cases**: Used in final grading (anti-cheat)
- Admin-managed via Test Case Manager component

🏗️ Technical Architecture

System Design

```
┌──────────────┐      ┌──────────────────┐      ┌──────────────┐
│   STUDENT    │──────▶│   REACT APP      │──────▶│   FIREBASE   │
│   Browser    │      │  (Frontend)      │      │  (Backend)   │
└──────────────┘      └──────────────────┘      └──────────────┘
        │                     │                      │
        │                     ▼                      ▼
        │             ┌──────────────────┐      ┌──────────────┐
        │             │   JUDGE0 API     │      │ VIOLATIONS   │
        │             │ (Code Executor)  │      │   LOGGING    │
        │             └──────────────────┘      └──────────────┘
        │
        └─ MediaDevices API (Screen Sharing)
```

Data Flow

Authentication Flow:

```
Login → Firebase Auth → Firestore Role Check → Dashboard Redirect
```

Exam Flow:

```
Code Validation → Screen Share → Fullscreen Activation
  ↓
Round 1 (Aptitude MCQ) → 30s Transition → Round 2 (Core MCQ)
  ↓
30s Transition → Round 3 (DSA Coding) → Submission
```

Code Execution Flow:

```
Student Writes Code → Judge0 Submission → Polling (1s intervals)
  ↓
Result Retrieved → Score Calculation → Firestore Storage
```

Application Routes

Route	Access	Description

Route	Access	Description
`/`	Public	Login page
`/register`	Public	Student/Admin registration
`/student`	Protected (Student)	Student dashboard with exam code entry
`/admin`	Protected (Admin)	Admin management console
`/exam/:examId`	Protected (Student)	Live exam interface (requires screen sharing)

🛡️ Security & Proctoring

Per-Round Violation System

Unique Feature: Each round has an independent 3-violation limit that resets between rounds.

```
┌─────────────────────────────────────────────────────────────┐
│        PER-ROUND VIOLATION TRACKING (3 MAX PER ROUND)        │
├─────────────────────────────────────────────────────────────┤
│                                                              │
│  Round 1: Violations 0/3 → 1/3 → 2/3 → 3/3 [AUTO-SUBMIT]    │
│                                                              │
│  ┌──────── 30-SECOND TRANSITION ────────┐                   │
│  │ ☑ Round 1 Complete                   │                   │
│  │ 🔄 Violations Reset to 0/3           │                   │
│  └──────────────────────────────────────┘                   │
│                                                              │
│  Round 2: Violations 0/3 → 1/3 [Fresh Counter]               │
│                                                              │
└─────────────────────────────────────────────────────────────┘
```

17+ Violation Detection Mechanisms

Fullscreen Violations (2 types)

- Fullscreen exit detection (`fullscreenchange` event)
- Escape key press blocking

Navigation Violations (2 types)

- Tab switching (`visibilitychange` event)
- Window blur (`blur` event)

Developer Tool Violations (3 types)

- F12 key press blocking
- Ctrl+Shift+I blocking
- Ctrl+Shift+J blocking

Clipboard Violations (5 types)

- Copy (Ctrl+C / Copy event)
- Paste (Ctrl+V / Paste event)
- Cut (Ctrl+X / Cut event)
- Select All (Ctrl+A)
- Text selection (CSS user-select: none)

System Violations (5 types)

- Save attempt (Ctrl+S)
- Print attempt (Ctrl+P)
- Screenshot attempt (PrintScreen key)
- Right-click (contextmenu event)
- Screen sharing stop detection

Violation Response System

Blocking Modal Approach:

1. Event detected → Check 1-second cooldown
2. Increment round counter (1/3, 2/3, 3/3)
3. Show full-screen blocking modal:
    - Round name badge
    - Violation count badge
    - Warning message
    - Visual progress bar (color-coded)
    - Remaining violations text
    - Acknowledge button (must click to continue)
4. Log to Firestore (userId, examId, type, timestamp, roundIndex)
5. Action: Continue (1-2 violations) OR Auto-submit (3rd violation)
6. Round transition: Reset counter to 0/3

Screen Sharing Enforcement

- **Mandatory full-screen sharing** using `getDisplayMedia` API
- **Surface validation**: Only entire screen accepted (rejects window/tab)
- **Continuous monitoring** every 5 seconds
- **Video track monitoring** for premature stop detection
- **SessionStorage verification** to prevent bypass
- **Automatic submission** if sharing stops during exam
- **Browser compatibility**: Chrome 72+, Edge 79+, Firefox 66+

🛠 Technology Stack

Frontend

```
Framework: React 19.2.5
Routing: React Router DOM 7.14.0
UI Components: Custom React Components
Icons: Lucide React 1.8.0
Styling: Native CSS (Modular + Inline)
Build Tool: Create React App 5.0.1
```

Backend & Services

```
Authentication: Firebase Auth 12.12.0
Database: Cloud Firestore (NoSQL)
Code Execution: Judge0 CE API
Hosting: Firebase Hosting / Vercel
```

Browser APIs

- **MediaDevices API** - Screen sharing (`getDisplayMedia`)
- **Fullscreen API** - Immersive exam mode
- **Page Visibility API** - Tab switch detection
- **Window Events API** - Blur/focus tracking
- **Clipboard API** - Copy/paste prevention
- **LocalStorage API** - Answer backup
- **SessionStorage API** - Screen share status

Development Tools

```
Package Manager: npm
Testing: React Testing Library
Linting: ESLint (react-app config)
Version Control: Git
```

External Services

- **Firebase Console** - Project configuration
- **Judge0 CE** - Remote code execution (https://ce.judge0.com)
- **Firestore** - Real-time database
- **Firebase Authentication** - User management

📥 Installation & Setup

Prerequisites

Ensure the following are installed on your system:

- **Node.js** (v16.0 or higher) - Download
- **npm** (v7.0 or higher) - Comes with Node.js
- **Git** - Download
- **Firebase Account** - Sign up
- **Modern Browser** - Chrome 72+, Edge 79+, or Firefox 66+

Installation Steps

Step 1: Clone Repository

```
git clone https://github.com/your-organization/placement-exam-system.git
cd placement-exam-system
```

Step 2: Install Dependencies

```
npm install
```

This installs all required packages:

- react, react-dom, react-router-dom
- firebase
- lucide-react
- testing utilities

Step 3: Firebase Project Setup

1. Go to Firebase Console
2. Click **"Add Project"** → Enter name: `placement-exam-system`
3. Disable Google Analytics (optional) → Click **"Create Project"**

Enable Authentication:

1. Firebase Console → **Authentication** → **"Get Started"**
2. Select **"Email/Password"** → Toggle **Enable** → **"Save"**

Create Firestore Database:

1. Firebase Console → **Firestore Database** → **"Create Database"**
2. Select **"Start in production mode"** → Choose location → **"Enable"**

Get Firebase Configuration:

1. Firebase Console → **Project Settings** (⚙ icon)
2. Scroll to **"Your apps"** → Click **Web icon** (</>)
3. Register app: `placement-exam-webapp`
4. Copy configuration object

Step 4: Configure Application

Create `src/firebase/config.js`:

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.appspot.com",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Step 5: Configure Firestore Security Rules

Firebase Console → **Firestore Database** → **Rules** → Paste:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAdmin() {
      return request.auth != null &&

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role ==
'admin';
    }

    function isAuthenticated() {
      return request.auth != null;
    }

    match /users/{userId} {
      allow read: if isAuthenticated();
      allow write: if request.auth.uid == userId || isAdmin();
      allow create: if isAuthenticated();
    }

    match /exams/{examId} {
      allow read: if isAuthenticated();
      allow write, create, delete: if isAdmin();
    }
```

```
      match /questions/{questionId} {
        allow read: if isAuthenticated();
        allow write, create, delete: if isAdmin();
      }

      match /submissions/{submissionId} {
        allow create, read: if isAuthenticated();
        allow update, delete: if isAdmin();
      }

      match /dsaSubmissions/{submissionId} {
        allow create, read: if isAuthenticated();
        allow update, delete: if isAdmin();
      }

      match /violations/{violationId} {
        allow create: if isAuthenticated();
        allow read, delete: if isAdmin();
      }

      match /adminActions/{actionId} {
        allow create, read: if isAdmin();
      }

      match /blockedStudents/{blockedId} {
        allow create, read: if isAdmin();
      }
    }
  }
```

Click **"Publish"**

Step 6: Start Application

```
npm start
```

Application opens at **http://localhost:3000**

📖 User Guide

For Students

1. Registration

1. Navigate to application URL
2. Click **"Register"**
3. Enter: Full Name, Email, Password (min 6 characters)
4. Click **"Register"**

5. Automatic role assignment: `student`

2. Taking an Exam

Pre-Exam:

1. Login with credentials
2. Read exam instructions thoroughly
3. Check **"I agree to the above terms"**
4. Enter exam code (provided by admin, case-insensitive)
5. Wait for countdown to reach zero

Screen Sharing:

1. Click **"Share Entire Screen & Start Exam"**
2. Browser prompts for screen sharing
3. **Select "Entire Screen"** (NOT window or tab)
4. Choose primary monitor → Click **"Share"**

During Exam:

Round 1: Aptitude (MCQ)

- Read questions and select one answer from 4 options
- Use Previous/Next buttons or click question numbers
- Clear Answer button to deselect
- Timer shows remaining time (MM:SS format)
- Violation counter displays in header (X/3)
- Auto-submit on: Time expiry OR 3rd violation

30-Second Transition Screen

- Shows "Round 1 Completed!" message
- Displays your score
- 30-second countdown with "Start Now" button
- Violations RESET to 0/3 for next round

Round 2: Core Subjects (MCQ)

- Same interface as Round 1
- Fresh violation counter (0/3)
- Technical questions on CS fundamentals

30-Second Transition #2

- Prepares for DSA coding round
- Violations reset again to 0/3

Round 3: DSA Coding

- Resizable interface (drag dividers to adjust layout)
- Problem description on left panel

- Code editor on right panel
- Select language from dropdown
- Write solution
- **Test Button**: Run visible test cases only (practice)
- **Submit Solution**: Run ALL tests (visible + hidden, final score)
- Navigate between multiple problems if configured
- Fresh violation counter (0/3)

Submission:

- Click **"Finish Round"** after attempting problems
- View comprehensive results:
  - Round-wise scores
  - Violation summary per round
  - Total score and percentage
  - Submission timestamp

For Administrators

1. Admin Account Setup

First Time:

1. Navigate to `/admin` route
2. Enter preset credentials:
   - Email: `admin@gmail.com`
   - Password: `admin123`
3. System auto-creates admin account with role: `admin`

2. Question Bank Management

Seed DSA Questions:

1. Admin Dashboard → **"Question Bank"** tab
2. Click **"🌱 Seed 5 DSA Questions"**
3. Confirm action
4. System adds 5 pre-built coding problems:
   - Two Sum (Easy)
   - Palindrome Number (Easy)
   - Fibonacci Sequence (Medium)
   - Maximum Subarray (Medium)
   - String Compression (Hard)

Add Custom Questions:

1. Select question type: MCQ or Coding
2. Fill required fields:
   - MCQ: Question text, 4 options, correct answer, category, difficulty
   - Coding: Title, description, examples, constraints, test cases, starter code

3. Click **"Add Question"**

3. Exam Creation

1. Admin Dashboard → **"Exams"** tab
2. Fill **"Create New Exam"** form:

```
Exam Title: "Placement Test 2026 - Batch 1"
Exam Code: "PLACE2026" (unique, alphanumeric)
Start Date: Select date
Start Time: Select time

Round Durations (minutes):
- Aptitude: 30
- Core: 45
- DSA: 60

Question Distribution:

Aptitude:
- Easy: 10
- Medium: 5
- Hard: 3

Core Subjects:
- Easy: 8
- Medium: 7
- Hard: 5

DSA (Coding):
- Easy: 1
- Medium: 2
- Hard: 1
```

3. Click **"Create Exam & Auto-Select Questions"**
4. System validates and creates exam
5. Share exam code with students

4. Monitoring Active Exams

Real-Time Monitor:

1. Click **"Monitoring"** tab
2. View:
   - Active students (last 5 minutes)
   - High violation alerts (3+ violations)
   - Recent submission feed
3. Admin actions available:
   - **Force Submit**: Immediately end student's exam

    - **Kick Student**: Block from continuing exam

5. Reviewing Results

1. Click **"Results"** tab
2. View all submissions with filters
3. Click **"View Details"** for specific submission
4. Review:
    - Round-wise scores
    - Time taken per round
    - Violation count per round
    - DSA code submissions with test results

👨‍💼 Admin Dashboard

Dashboard Sections

1. Overview Tab

- **System Statistics**:
    - Total exams created
    - Active students count
    - Total submissions
    - Question bank status
- **Quick Actions**: Create Exam, Add Question, View Results, Monitor

2. Question Bank Tab

- **Status Boxes**:
    - 🟢 Aptitude Questions (Easy/Medium/Hard counts)
    - 🟢 Core Subject Questions (Easy/Medium/Hard counts)
    - 🟢 DSA Coding Questions (Easy/Medium/Hard counts)
- **Actions**: Seed DSA Questions, Add Question, Delete Questions
- **Question List**: All questions with category, difficulty, options
- **Test Case Manager**: Edit visible/hidden test cases for DSA questions

3. Exams Tab

- **Create Exam Form**: Full configuration wizard
- **Exam List**: All created exams with:
    - Title, Code, Start Time, Durations
    - Question counts per round
    - Actions: View, Edit, Delete

4. Monitoring Tab (Real-Time)

- **Live Feed** (auto-refresh via Firestore onSnapshot):

    - Active students list
    - High violation alerts
    - Recent submissions
- **Admin Controls**:
    - Force submit student
    - Kick student
    - View violation details

5. Results Tab

- **Submission Table**:
    - Columns: Student, Email, Exam, Score, Violations, Time
    - Filters: By exam, date range, score range
- **Details View**:
    - Round-wise breakdown
    - Time taken per round
    - Violation log with timestamps
    - DSA code viewer with execution results

🗄️ Database Architecture

Firestore Collections

1. users

```
{
  uid: "firebase_auth_uid",
  name: "John Doe",
  email: "john@example.com",
  role: "student" | "admin",
  createdAt: Timestamp
}
```

2. questions

MCQ Question:

```
{  id: "auto_generated",
  type: "mcq",
  category: "Aptitude" | "Core Subjects",
  difficulty: "Easy" | "Medium" | "Hard",
  text: "What is 2+2?",
  options: ["2", "3", "4", "5"],
  correct: 2, // index
  points: 10,
```

```
  createdAt: Timestamp
}
```

DSA Coding Question:

```
{
  id: "auto_generated",
  type: "coding",
  category: "DSA",
  difficulty: "Easy" | "Medium" | "Hard",
  title: "Two Sum",
  description: "Given an array of integers...",
  examples: [
    {
      input: "[2,7,11,15], target=9",
      output: "[0,1]",
      explanation: "nums[0] + nums[1] == 9"
    }
  ],
  constraints: [
    "2 <= nums.length <= 10^4",
    "-10^9 <= nums[i] <= 10^9"
  ],
  testCases: [
    {
      input: "[2,7,11,15]\n9",
      expectedOutput: "[0,1]",
      explanation: "Basic case",
      hidden: false // Visible test
    },
    {
      input: "[3,3]\n6",
      expectedOutput: "[0,1]",
      explanation: "Duplicate elements",
      hidden: true // Hidden test
    }
  ],
  starterCode: {
    python: "def twoSum(nums, target):\n    pass",
    cpp: "vector<int> twoSum(vector<int>& nums, int target) {\n}",
    c: "int* twoSum(int* nums, int numsSize, int target, int* returnSize) {\n}",
    java: "public int[] twoSum(int[] nums, int target) {\n}",
    javascript: "function twoSum(nums, target) {\n}"
  },
  defaultLanguage: "python",
  timeLimit: 2, // seconds
  memoryLimit: 256, // MB
  points: 100,
  createdAt: Timestamp
}
```

3. exams

```
{
  id: "auto_generated",
  title: "Placement Test 2026",
  examCode: "place2026",
  createdBy: "admin_uid",
  startTime: Timestamp,
  roundDurations: {
    aptitude: 30,
    core: 45,
    dsa: 60
  },
  questionSet: [
    // Array of selected questions with type (mcq/coding)
  ],
  createdAt: Timestamp
}
```

4. submissions

```
{
  id: "auto_generated",
  userId: "student_uid",
  userEmail: "student@example.com",
  examId: "exam_id",
  examTitle: "Placement Test 2026",
  scores: [
    { round: "Round 1: Aptitude", score: 15, total: 18 },
    { round: "Round 2: Core Subjects", score: 18, total: 20 },
    { round: "Round 3: DSA", score: 75, total: 100 }
  ],
  totalScore: 108,
  totalQuestions: 138,
  violations: [1, 0, 2], // Per-round: [Round1, Round2, Round3]
  submittedAt: Timestamp,
  reason: "manual" | "auto" | "violations" | "time_expired"
}
```

5. dsaSubmissions

```
{
  id: "auto_generated",
  userId: "student_uid",
  examId: "exam_id",
  questionId: "question_id",
  questionTitle: "Two Sum",
```

```
    code: "def twoSum(nums, target):\n    ...",
    language: "python",
    languageId: 71,
    testResults: [
      {
        passed: true,
        input: "[2,7,11,15]\n9",
        output: "[0,1]",
        expected: "[0,1]",
        time: "0.001",
        memory: 2048,
        hidden: false
      }
    ],
    passedTests: 6,
    totalTests: 7,
    score: 85.71,
    submittedAt: Timestamp
  }
```

6. violations

```
  {
    id: "auto_generated",
    userId: "student_uid",
    userEmail: "student@example.com",
    examId: "exam_id",
    violationType: "Tab switching is not allowed...",
    timestamp: Timestamp,
    roundIndex: 0, // 0=Round1, 1=Round2, 2=Round3
    roundViolations: 2 // Count for THIS round
  }
```

7. adminActions

```
  {
    id: "auto_generated",
    action: "force_submit" | "kick_student",
    targetUserId: "student_uid",
    targetEmail: "student@example.com",
    examId: "exam_id",
    timestamp: Timestamp,
    adminEmail: "admin@example.com"
  }
```

8. blockedStudents

```
{
  id: "auto_generated",
  userId: "student_uid",
  userEmail: "student@example.com",
  examId: "exam_id",
  blockedAt: Timestamp,
  reason: "admin_kicked",
  adminEmail: "admin@example.com"
}
```

🚀 Deployment

Option 1: Firebase Hosting

```
# Install Firebase CLI
npm install -g firebase-tools
firebase login

# Initialize Firebase
firebase init
# Select: Hosting, Firestore
# Build directory: build
# Single-page app: Yes

# Build and Deploy
npm run build
firebase deploy
```

Live at: `https://your-project.web.app`

Option 2: Vercel Deployment

```
# Install Vercel CLI
npm install -g vercel
vercel login

# Deploy
npm run build
vercel --prod
```

Live at: `https://your-project.vercel.app`

Production Checklist

- ☑ Update Firebase security rules
- ☑ Deploy Firestore indexes

- ☑ Configure environment variables
- ☑ Test screen sharing on HTTPS
- ☑ Set up error monitoring
- ☑ Configure backup strategy
- ☑ Enable Firebase Analytics (optional)
- ☑ Set up custom domain (optional)

⚡ Performance & Scalability

Capacity

- **Concurrent Users**: 500+ simultaneous exam takers
- **Database Operations**: Optimized with Firestore indexes
- **Code Execution**: Judge0 free tier (50 req/day) or paid tier
- **Response Time**: < 2 seconds page load, < 5 seconds code execution

Optimization Techniques

Frontend

- React lazy loading for route components
- useMemo and useCallback for expensive operations
- LocalStorage for answer backup
- Debouncing for Firestore writes (2-second delay)

Backend

- Firestore composite indexes
- Security rules for access control
- onSnapshot for real-time updates (admin only)
- Batch operations for bulk deletions

Code Execution

- Asynchronous Judge0 polling
- 30-second timeout with graceful failure
- Result caching to avoid duplicate API calls

📊 Project Statistics

Metric	Value
Programming Languages	5 (Python, C, C++, Java, JavaScript)
Total Routes	5
Firestore Collections	8

Metric	Value
React Components	15+
Lines of Code	~6,000+
Pre-built DSA Questions	5 (with 36 test cases)
Violation Detection Types	17+
Supported Browsers	Chrome 72+, Edge 79+, Firefox 66+
Max Concurrent Users	500+
Auto-save Interval	2 seconds
Violation Cooldown	1 second
Round Transition Time	30 seconds

📚 Support & Documentation

Technical Documentation

- **React**: https://react.dev
- **Firebase**: https://firebase.google.com/docs
- **Judge0 API**: https://ce.judge0.com
- **Firestore**: https://firebase.google.com/docs/firestore

Project Files

- `FEATURES.md` - Complete feature list
- `DEPLOYMENT_GUIDE.md` - Detailed deployment instructions
- `DSA_ROUND_GUIDE.md` - DSA question setup
- `JUDGE0_INTEGRATION.md` - Judge0 API reference
- `FIRESTORE_SETUP.md` - Database configuration

Browser Compatibility

Browser	Version	Status
Chrome	72+	☑ Fully Supported (Recommended)
Edge	79+	☑ Fully Supported
Firefox	66+	☑ Fully Supported
Safari	13+	⚠ Partial Support (screen sharing limited)

Common Issues & Solutions

Q: Screen sharing not working?

- Ensure using Chrome/Edge/Firefox (latest version)

- Grant screen recording permissions at OS level
- Use HTTPS in production (localhost works for dev)

Q: Code execution fails?

- Check Judge0 API status: https://status.judge0.com
- Verify internet connection
- Free tier limit: 50 requests/day (upgrade if needed)

Q: Exam code not accepted?

- Codes are case-insensitive
- Maximum 3 attempts allowed
- Verify exam startTime is set correctly

Q: Questions not showing?

- Seed questions via Admin Dashboard
- Check Firestore security rules are published
- Verify category matches exam configuration

License & Attribution

License: Educational/Academic Use
Copyright: © 2026 Team Kairo - All Rights Reserved

Terms of Use

- Free for academic and educational institutions
- Attribution to Team Kairo appreciated
- Contributions welcome via GitHub
- No warranty or liability provided

Contributing

Contributions are welcome! Priority development areas:

Wanted Features

- Code plagiarism detection (MOSS/JPlag)
- Video proctoring with webcam recording
- Resume interrupted exams functionality
- Bulk question upload (CSV/JSON import)
- Email notifications (exam reminders, results)
- Advanced analytics dashboard
- Export results to PDF/Excel

Bug Reports

Include: Browser version, OS, reproduction steps, screenshots, console errors

---

👥 Development Team

Team Kairo 🏆

A dedicated team focused on creating innovative educational technology solutions for fair and comprehensive student assessment.

🎉 Acknowledgments

Special thanks to:

- **Firebase Team** - Backend infrastructure and authentication
- **Judge0** - Remote code execution engine
- **Microsoft** - Monaco Editor
- **React Team** - Frontend framework
- **Open Source Community** - Tools, libraries, and inspiration

---

📈 Version History

v2.0.0 (April 2026) - Current Release ✨

- ☑ Complete DSA coding environment (Round 3)
- ☑ Judge0 API integration with 5 languages
- ☑ Per-round violation system (3 max per round)
- ☑ Real-time admin monitoring
- ☑ Test case management system
- ☑ Auto-grading with hidden tests
- ☑ Resizable code editor interface

v1.5.0 - Security & Proctoring

- ☑ Mandatory screen sharing enforcement
- ☑ 17+ violation detection mechanisms
- ☑ Fullscreen enforcement
- ☑ Blocking violation modals

v1.0.0 - Initial Release

- ☑ Firebase Authentication
- ☑ Two-round MCQ exam system
- ☑ Admin dashboard
- ☑ Basic results management

Status: ☑ Production Ready
Last Updated: April 13, 2026

Developed By: Team Kairo 🏆

🎓 **Empowering Educational Excellence Through Technology** 🚀

For technical support or inquiries, please contact Team Kairo